1 Zero-free multiplicative carrier interface

Source: `PrimeTensor/Structure.lean`

What this file establishes

By the end of `PrimeTensor/Algebra.lean` two carriers — MulRat and its completion MulReal — satisfied the same six laws, each proved twice, once on each side. This file introduces the interface that names those six laws, records both carriers as instances of it, and proves the first theorem of the development that is stated once and holds for every carrier at once. It closes by checking that the embedding ofRat : MulRat → MulReal respects all of the structure.

The interesting content is a negative decision, and the module header states it outright: Lean's Group and CommGroup are *not* used, and the class defined instead is deliberately weaker than they are. Why a weaker class is the right one here is the subject of Design note 1.2.

1.1 The interface

Definition 1.1 (`IsMulCarrier`)**.** For a type G already equipped with One, Mul and Inv instances, IsMulCarrier(G) is the proposition asserting the six laws

$$\begin{aligned} (ab)c = a(bc), \qquad & 1 \cdot a = a, \qquad & a \cdot 1 = a, \\ ab = ba, \qquad & (a^{-1})^{-1} = a, \qquad & a \cdot a^{-1} = 1. \end{aligned}$$

```
class IsMulCarrier (G : Type*) [One G] [Mul G] [Inv G] : Prop where
  mul_assoc : ∀ a b c : G, (a * b) * c = a * (b * c)
  one_mul : ∀ a : G, 1 * a = a
  mul_one : ∀ a : G, a * 1 = a
  mul_comm : ∀ a b : G, a * b = b * a
  inv_inv : ∀ a : G, (a⁻¹)⁻¹ = a
  mul_inv : ∀ a : G, a * a⁻¹ = 1
```

Three features of the declaration carry the design.

It is a *mixin*: the three operations appear as instance *parameters* in square brackets, not as fields. A type does not acquire a multiplication by being an IsMulCarrier; it must already have one, and the class only asserts that the one it has obeys the laws. So no competing One, Mul or Inv instance is ever created, and there is no possibility of two multiplications on the same carrier disagreeing about which is meant.

It is Prop-*valued*. Every field is an equation, so an IsMulCarrier instance carries no data whatever — nothing that could be computed with, and nothing that could be unfolded to something unexpected.

And it is *exactly six laws*, closed under nothing. There is no division, no power operation, no additive field, and no cancellation axiom.

Design note 1.2 (Why not CommGroup). The six laws say that G is an abelian group, so CommGroup G would be provable and would bring the whole library with it. The header rejects it for the operations it would drag in.

Lean's Monoid carries a natural-power field npow : $\mathbb{N} \to M \to M$, and DivInvMonoid carries an integer power zpow : $\mathbb{Z} \to G \to G$ and a division Div. They are fields with defaults, not

theorems: a CommGroup instance on MulReal would equip the carrier with $a^0 = 1$, with a^{-n}, and with a/b, whether or not any of them is ever written.

Each is precisely what this development has been at pains to exclude. Depth of `PrimeTensor/Depth.lean` has no zeroth constructor, so that every axis is nonempty and every fold needs no identity element; a npow defined at 0 reintroduces the zeroth index at the level of the carrier and makes the exclusion cosmetic. Div reintroduces a second primitive operation on top of Inv, so that a statement can be written in two inequivalent-looking ways and every downstream simp set has to decide between them. And an integer power presupposes $\mathbb{Z}$, an additive object, in the object language of a development whose stated constraint is that nothing additive appears.

The mixin is the smallest interface that gives the laws without the operations. Its cost is real and was visible in `PrimeTensor/Scale/Laws.lean` and `PrimeTensor/Algebra.lean`: since ac_rfl, ring and group all dispatch on the library's algebraic classes and not on this one, associativity-commutativity regroupings still have to be written out by hand, and both files did so. The trade is spelled-out rewriting in exchange for a carrier that carries no operation nobody asked for.

Lemma 1.3 (`IsMulCarrier.inv_mul`)**.** *In any* IsMulCarrier*,* $a^{-1} \cdot a = 1$*.*

```
theorem inv_mul (a : G) : a⁻¹ * a = 1 := by
  rw [IsMulCarrier.mul_comm]
  exact IsMulCarrier.mul_inv a
```

Proof. Commute and apply the sixth law. □

Remark 1.4 (The first generic theorem)*.* Trivial as it is, this is the first statement in the development proved once for all carriers rather than once per carrier. The same fact was proved separately as MulRat.inv_mul in `PrimeTensor/Scale/Laws.lean` and as MulReal.inv_mul in `PrimeTensor/Algebra.lean`, each by this identical two-line argument. Both survive: nothing is deleted, and the concrete names remain the ones the earlier files' simp sets refer to. What changes is that a *third* carrier would now inherit the lemma instead of re-proving it.

Remark 1.5 (The axiom list is not minimal)*.* inv_inv follows from the other five. Inverses in a monoid are unique — if $xy = 1$ and $xz = 1$ then $y = y \cdot 1 = y(xz) = (yx)z = (xy)z = 1 \cdot z = z$, using commutativity for the fourth step — and both a and $(a^{-1})^{-1}$ are inverses of a^{-1}, the first by Lemma 1.3 and the second by the sixth law applied to a^{-1}. Listing it as an axiom costs one line per instance and saves that derivation at every use, which for a class whose only two instances already had the fact proved is the cheaper side of the trade.

1.2 The two instances

Proposition 1.6 (`IsMulCarrier MulRat` and `IsMulCarrier MulReal`)**.** *Both* MulRat *and* MulReal *are* IsMulCarrier*s.*

Proof. Each instance is a six-field record whose entries are the correspondingly named theorems already proved: MulRat.mul_assoc, one_mul, mul_one, mul_comm, inv_inv, mul_inv from `PrimeTensor/MulRat.lean`, and the six theorems of the same names on MulReal from `PrimeTensor/Algebra.lean`. Nothing is proved here; the file only records that what was proved has the shape the interface asks for. □

That both instances are literally lists of existing names is the evidence that the interface was extracted from the practice rather than imposed on it: the six laws are exactly the six that both carriers had already been given, in the same order and with the same statements.

1.3 The embedding respects the structure

Lemma 1.7 (`MulReal.ofRat_one`, `ofRat_mul` and `ofRat_inv`)**.** $\mathrm{ofRat}(1) = 1$*, and for all* $a, b :$ MulRat*,*

$$\mathrm{ofRat}(a \cdot b) = \mathrm{ofRat}(a) \cdot \mathrm{ofRat}(b), \qquad \mathrm{ofRat}(a^{-1}) = \mathrm{ofRat}(a)^{-1}.$$

```
@[simp] theorem ofRat_mul (a b : MulRat) :
    ofRat (a * b) = ofRat a * ofRat b := by
  change
    Quotient.mk MulAsymptotic.setoid (MulCauchyStream.constant (a * b)) =
      Quotient.mk MulAsymptotic.setoid
        (MulCauchyStream.mul
          (MulCauchyStream.constant a)
          (MulCauchyStream.constant b))
  apply Quotient.sound
  apply MulAsymptotic.of_pointwise
  intro n
  rfl
```

Proof. The first is rfl: MulReal.one was *defined* as $\mathrm{ofRat}(1)$ in `PrimeTensor/Completion.lean` and installed as the One instance, so the two sides are the same term.

The other two are the four-line shape of `PrimeTensor/Algebra.lean`: reduce the equality of classes to an asymptotic equivalence by Quotient.sound, reduce that to a termwise equality by of_pointwise, and observe that the termwise equality is rfl. It is rfl because the n-th term of the constant stream at ab is ab, and the n-th term of the product of the constant streams at a and at b is also ab — the two streams are not merely asymptotic, they are equal, and the same holds for inversion. □

Remark 1.8 (Predicted, and for the predicted reason)*.* `PrimeTensor/Algebra.lean` closed by noting that the homomorphism property of ofRat was not yet stated but would follow from of_pointwise in one line, since the constant stream of a product is the product of the constant streams. That is exactly the proof above, and the "one line" is the final rfl. No analysis enters: the levels and anchors of the completion play no part, because constant streams agree identically rather than eventually.

Remark 1.9 (Injectivity is still missing)*.* ofRat is now known to preserve the pivot, the multiplication and the inversion, which is everything the interface names. It is still not known to be injective, and injectivity is not a formal consequence of being a homomorphism — it is the statement that two rationals whose constant streams are close at every level are equal, which is an archimedean fact about MulRat and needs an argument about the ladder. Until it is proved, MulReal is a carrier receiving a structure-preserving map from MulRat, not yet an extension of it.

Remark 1.10 (The mixin is not yet used generically)*.* Lemma 1.3 is the only theorem stated for a general IsMulCarrier, and no later result in this file is. The interface's payoff is entirely deferred: it exists so that constructions downstream can be written once against the laws instead of twice

against the two carriers. What the file delivers now is the declaration, the two instances, and the evidence that the embedding between them is a morphism of the structure the interface describes.

Summary of the correspondence

Lean declaration	Kind	Here
`IsMulCarrier`	class	Def. 1.1
`IsMulCarrier.inv_mul`	theorem	Lem. 1.3
`IsMulCarrier MulRat`	instance	Prop. 1.6
`IsMulCarrier MulReal`	instance	Prop. 1.6
`MulReal.ofRat_one`	simp thm	Lem. 1.7
`MulReal.ofRat_mul`	simp thm	Lem. 1.7
`MulReal.ofRat_inv`	simp thm	Lem. 1.7

Every declaration of `PrimeTensor/Structure.lean` appears above, together with the six fields of the class, listed in Definition 1.1. The file contains no `sorry`, no axiom, and no unproved named proposition. Remarks 1.4, 1.5, 1.8, 1.9 and 1.10 are stated for the reader and are not declarations of the file; the derivation in Remark 1.5 is not carried out in Lean.